Week 1, Day 4: Multithreading Basics and Two Pointers

Prepared for Upendra Kumar Sharma | 3–4 hours | Main topic: Threads, synchronized, volatile, happens-before | DSA: Two pointers

Today's Schedule

- **0:00–0:40** Study thread basics and the thread lifecycle (sections 1 and 2).
- **0:40–1:30** Study synchronized, volatile, atomics and the Java Memory Model (sections 3 to 5).
- **1:30–1:50** Study wait/notify and write the producer-consumer code from section 6 yourself.
- **1:50–2:50** DSA: solve the 3 two-pointer problems (section 7). Spend at most 25 minutes per problem before reading the hint.
- **2:50–3:20** Answer the interview questions aloud (section 8) and add 10 flashcards.

1. Creating and Running Threads

- **Thread subclass:** extend Thread and override run(). Rarely preferred, because it ties the task to the thread.
- **Runnable:** implement run(), no return value, cannot throw checked exceptions. Preferred over extending Thread.
- **Callable and Future:** call() returns a value and may throw checked exceptions. Submit it to an ExecutorService and read the result with future.get().
- **start() vs run():** start() creates a new thread that runs run(). Calling run() directly just executes it on the current thread. Calling start() twice throws IllegalThreadStateException.
- **Daemon threads:** background threads that do not stop the JVM from exiting (the garbage collector is one). Set with setDaemon(true) before start().
- In real applications you rarely create raw threads. You use executors (Day 5), but interviewers still test the basics.

2. Thread Lifecycle

- **NEW:** created, start() not yet called.
- **RUNNABLE:** running or ready to run on the CPU.

- **BLOCKED:** waiting to acquire a monitor lock (entering a synchronized block).
- **WAITING:** waiting indefinitely, after Object.wait(), Thread.join() or LockSupport.park().
- **TIMED_WAITING:** waiting with a timeout, after sleep(ms), wait(ms) or join(ms).
- **TERMINATED:** run() has finished.

Key differences

- **sleep() vs wait():** sleep() is a Thread method and keeps any lock it holds. wait() is an Object method, must be called inside synchronized, and releases the monitor while waiting.
- **join():** makes the current thread wait until another thread finishes.
- **interrupt():** sets the interrupt flag. Blocking methods such as sleep and wait throw InterruptedException. Handle it by restoring the flag (Thread.currentThread().interrupt()) or exiting cleanly, never by silently swallowing it.

3. The Problems Threads Create

- **Race condition:** the result depends on the timing of threads. Example: two threads doing count++ on a shared field.
- **Visibility:** one thread's write is not seen by another, because values can sit in CPU caches or registers.
- **Reordering:** the compiler and CPU may reorder instructions as long as a single thread cannot notice.
- **count++ is not atomic.** It is three steps: read, add, write. Two threads can both read the same value and lose an update.

4. synchronized

- Uses the **intrinsic lock (monitor)** of an object. Only one thread at a time can hold it, and the lock is **reentrant**, meaning the same thread can enter again.
- **Synchronized instance method:** locks on this. **Static synchronized method:** locks on the Class object. These are two different locks.
- **Synchronized block:** locks on the object you name, so you can keep the critical section small. Prefer a private final lock object over this, so outside code cannot interfere with your lock.
- It gives you **both mutual exclusion and visibility**. Everything written before releasing the lock is visible to the next thread that acquires the same lock.
- **Cost:** contention causes blocking and context switches. Keep synchronized regions short and never call unknown code or do I/O while holding a lock.

```
public class Counter {
    private final Object lock = new Object();
    private int count;

    public void increment() {
        synchronized (lock) {
            count++;
        }
    }
    public int get() {
        synchronized (lock) {
            return count;
        }
    }
}
```

5. volatile, Atomics and the Java Memory Model

volatile

- A write to a volatile field is visible to any thread that later reads it, and instructions are not reordered across it.
- It does **not** make compound actions atomic. volatile int count with count++ is still a race.
- Good uses: stop flags, status flags, safe publication of an immutable object, and the double-checked locking singleton.

Atomic classes

- AtomicInteger, AtomicLong, AtomicReference use **CAS (compare-and-swap)** at the CPU level. They are lock-free and make single-variable updates atomic.
- Under heavy contention, LongAdder performs better than AtomicLong for counters.
- CAS can suffer from the ABA problem. AtomicStampedReference addresses it.

happens-before rules (Java Memory Model)

If action A happens-before action B, then A's results are visible to B. The main rules:

1. **Program order:** each action in a thread happens-before later actions in the same thread.
2. **Monitor lock:** unlocking a monitor happens-before every later lock of that same monitor.
3. **Volatile:** a write to a volatile field happens-before every later read of that field.
4. **Thread start:** Thread.start() happens-before any action in the started thread.

5. **Thread join:** all actions in a thread happen-before another thread returns from join() on it.
6. **Final fields:** correctly constructed objects publish their final fields safely to other threads.

Double-checked locking singleton (volatile is required)

```
public class Config {
    private static volatile Config instance;
    private Config() {}

    public static Config getInstance() {
        if (instance == null) {
            synchronized (Config.class) {
                if (instance == null) {
                    instance = new Config();
                }
            }
        }
        return instance;
    }
}
```

Without volatile, another thread could see a non-null reference to a partially constructed object because of instruction reordering. A simpler alternative is the holder class idiom or an enum singleton.

6. wait, notify and Producer-Consumer

- wait(), notify() and notifyAll() must be called while holding the object's monitor, otherwise IllegalMonitorStateException is thrown.
- **Always call wait() inside a while loop** that rechecks the condition, because of spurious wakeups and because another thread may have consumed the condition first.
- notify() wakes one waiting thread, notifyAll() wakes all. Use notifyAll() unless you are sure only one waiter can proceed.
- In production code, prefer BlockingQueue, which does this for you. Interviewers still ask for the manual version.

```
class BoundedBuffer<T> {
    private final Queue<T> queue = new LinkedList<>();
    private final int capacity;

    BoundedBuffer(int capacity) { this.capacity = capacity; }

    public synchronized void put(T item) throws InterruptedException {
        while (queue.size() == capacity) wait();
        queue.add(item);
        notifyAll();
    }
    public synchronized T take() throws InterruptedException {
        while (queue.isEmpty()) wait();
        T item = queue.remove();
        notifyAll();
        return item;
    }
}
```

7. DSA: Two Pointers (3 Problems)

Problem 1: 3Sum

- **Idea:** sort the array. Fix index i, then use two pointers (left = i + 1, right = end) to find pairs that sum to -nums[i]. Move left up when the sum is too small, right down when it is too large.
- **Avoid duplicates:** skip repeated values for i, and after finding a triplet skip repeated values for left and right.
- **Complexity:** O(n^2) time, O(1) extra space (ignoring the sort).
- **Edge cases:** fewer than 3 elements, all zeros, all positive numbers (stop early when nums[i] > 0).

Problem 2: Container With Most Water

- **Idea:** start with pointers at both ends. The area is min(height[l], height[r]) x (r - l). Move the pointer at the shorter line inward, because moving the taller one can never increase the area.
- **Complexity:** O(n) time, O(1) space.
- **Edge cases:** two elements, equal heights, zeros.

```
public int maxArea(int[] height) {
    int l = 0, r = height.length - 1, best = 0;
    while (l < r) {
        best = Math.max(best, Math.min(height[l], height[r]) * (r - l));
        if (height[l] < height[r]) l++; else r--;
    }
    return best;
}
```

Problem 3: Trapping Rain Water

- **Idea:** the water above a bar is min(maxLeft, maxRight) - height. Use two pointers with running leftMax and rightMax. Process the side with the smaller max, because that side's water level is already determined.
- **Complexity:** O(n) time, O(1) space. The alternative with two precomputed arrays is O(n) space.
- **Edge cases:** fewer than 3 bars, strictly increasing or decreasing heights (answer 0).

```
public int trap(int[] height) {
    int l = 0, r = height.length - 1, leftMax = 0, rightMax = 0, water = 0;
    while (l < r) {
        if (height[l] < height[r]) {
            if (height[l] >= leftMax) leftMax = height[l];
            else water += leftMax - height[l];
            l++;
        } else {
            if (height[r] >= rightMax) rightMax = height[r];
            else water += rightMax - height[r];
            r--;
        }
    }
    return water;
}
```

8. Interview Questions to Answer Aloud

1. Runnable vs Callable. When do you use each?
2. What is the difference between start() and run()?
3. Explain the thread states and what moves a thread between them.
4. sleep() vs wait(): which releases the lock?
5. What does synchronized guarantee? What is the difference between a synchronized method and a synchronized block?

6. Static synchronized vs instance synchronized: do they block each other?
7. What does volatile do, and why is volatile count++ still unsafe?
8. Explain happens-before with two examples.
9. Why is the double-checked locking singleton broken without volatile?
10. Why must wait() be called in a loop?
11. notify vs notifyAll: which one is safer and why?
12. AtomicInteger vs synchronized: how does CAS work, and when does it perform worse?
13. How do you stop a thread safely? Why is Thread.stop() deprecated?

9. End-of-Day Checklist

- [ ] I can draw the thread state diagram and name what triggers each transition.
- [ ] I can explain what synchronized and volatile each guarantee, and what they do not.
- [ ] I can write the double-checked locking singleton and the bounded buffer from memory.
- [ ] I can list at least four happens-before rules.
- [ ] I solved all 3 two-pointer problems and can state their complexities.
- [ ] I wrote at least 10 flashcards from questions I got wrong.

Tomorrow (Day 5): ExecutorService, thread pool sizing, ReentrantLock, deadlock and ThreadLocal, plus sliding window problems.